CITS5508: Assignment 1

Semester 1, 2026

Assessed, worth **20%**. Due **11:59 pm Friday 17th April 2026**.

1 Outline

This assignment is divided into two parts: a *classification* and a *regression* task. Detailed descriptions of each task are given below.

While this assignment is centred around programming, you should treat your submission as a report. A specific set of tasks is given for both **Part 1**, and **Part 2** however, your assignment is to perform an investigation. Do not expect to receive full marks for one-word answers when asked to comment on your results. Explain your thoughts in detail.

2 Submission

- You should submit your assignment via LMS. The submission should be a single Jupyter Notebook file (with an `.ipynb` extension).
- Your notebook should include all code, outputs, descriptions and comments. Treat it as a report – introduce the topic, describe your methodology, present your results and conclusions.
  - Make use of Markdown cells inserted at appropriate places to explain your code. In addition to Markdown cells, some short comments can be put alongside the Python code
  - Dividing the assignment into suitable sections and subsections (with section and subsection numbers and meaningful headings) would make your portfolio easier to follow.
  - Please see `sample.ipynb` under the Week 2 content on LMS for an example of how to structure your notebook.
- Your notebook should already include all output, but must also run without errors when using the CITS5508 Anaconda environment. Note that Google Colaboratory will produce identical outputs to our environment.

2.1 AI use in this assignment

The AI policy applied to this assignment is based on UWA's **Tier 2: AI assistance or collaboration** guidelines. For more information, see UWA's AI Assessment Guidelines.

- You may use AI tools to assist you in completing this assignment, but no AI-generated content is allowed in the final submission.
- Any descriptions, explanations, comments, code, or outputs in your final submission must be in your own words, and you must be able to justify and explain them.
- The correctness and quality of your submission is your responsibility. You should not rely on AI tools to generate content for you, and you should carefully review any material that you include in your final submission.

3 Part 1: Softmax Regression

Your first task is to implement a Softmax Regression classifier from scratch using gradient descent optimisation, and apply it to the MNIST dataset.

Recall from the lecture that Softmax Regression is a generalisation of logistic regression to the multiclass case. It estimates the probability of $\mathbf{x}$ belonging to class k as,

$$\hat{p}_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^{K} \exp(s_j(\mathbf{x}))},$$

where K is the total number of classes and $s_k(\mathbf{x}) = (\theta^{(k)})^\intercal \mathbf{x}$.

The cost function for Softmax Regression is the cross-entropy loss, which is given by,

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right),$$

where $y_k^{(i)}$ is the target probability that the ith instance belongs to class k. Then $\partial J / \partial \boldsymbol{\theta}^{(k)}$ is given by,

$$\frac{\partial J}{\partial \boldsymbol{\theta}^{(k)}} = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{p}_k^{(i)} - y_k^{(i)}\right) \mathbf{x}^{(i)}.$$

1. Load the MNIST dataset from OpenML using `scikit-learn`:

```
from sklearn.datasets import fetch_openml
X, y = fetch_openml("mnist_784", version=1, return_X_y=True, as_frame=False)
```

2. Display some examples from the dataset. Describe what the features and labels represent.
3. Split the dataset into a training, validation, and test set. You should use 70% of the data for training, and 15% for validation, and 15% for testing.

   Hint: you can use `sklearn.model_selection.train_test_split` to do this.

   (a) Display the class distribution in the training, validation and test sets.

   (b) Do you need to stratify your split? Why or why not?
4. Implement softmax regression with gradient descent using vectorised NumPy code. Implement early stopping based on the validation set. Do not use `scikit-learn` for this task – only NumPy and basic Python.

   (a) Display a plot showing how the training and validation loss changes over time for your model.

   (b) What do you observe? Is this what you expected?
5. Fit a `sklearn.linear_model.LogisticRegression` model to the data. Do not apply regularisation. Compare the performance of your softmax regression model to the `scikit-learn` model.

   (a) Display a confusion matrix for each model.

   (b) Show appropriate evaluation metrics for both models.

   (c) Comment on the results. Are they what you expected? Why or why not?
6. Comment on how the performance of your softmax regression model compares to the logistic regression model from `scikit-learn`.

4 Part 2: Support Vector Machine Regression

Your second task is to investigate how different kernels affect the performance of Support Vector Machine (SVM) regression on a synthetic dataset.

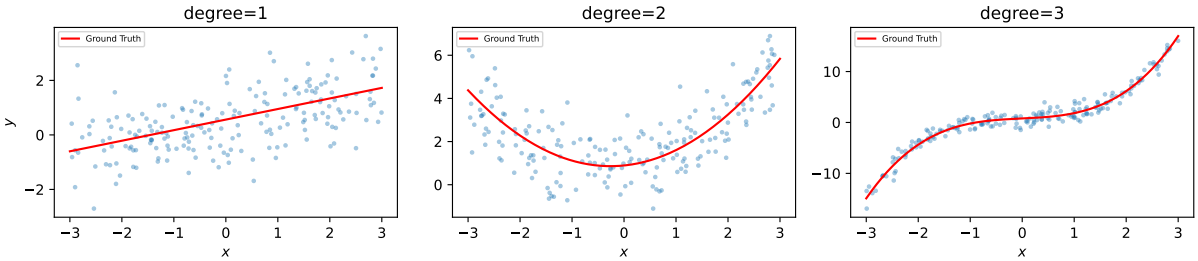

Figure 1: Example synthetic polynomial datasets of different degrees.

You will generate your own toy datasets using polynomial functions, and then fit SVM regression models with different kernels to the data.

1. Write a function that can generate a random toy n-degree polynomial dataset of the form,

$$y = \sum_{k=0}^{n} a_k x^k + \epsilon,$$

   where $x \in [-3, 3]$, $a_k \sim \mathcal{U}(0, 1)$, and $\epsilon \sim \mathcal{N}(0, 1)$. Your function should take two arguments: `degree`, and `n_samples`. Show your function works by plotting some generated datasets.

2. Generate three toy datasets using your function: a linear, a quadratic, and cubic dataset. Use `n_samples = 1000`. Make sure you have a training and test set for each dataset.

3. Experiment with fitting a `sklearn.svm.SVR` model to each of the three datasets using `kernel='linear'`, `kernel='poly'` and `kernel='rbf'`.

   (a) Show plots of the fitted regression function for each kernel and each dataset.

   (b) Comment on the results. Are they what you expected?

4. Perform grid-search with k-fold cross-validation ($k = 5$) on at least three hyperparameters (including kernel) with two different values each ($2^3 = 8$ total combinations) for your cubic dataset.

   (a) What hyperparameters did you choose? How many models did you fit in total?

   (b) Present your results for the optimal hyperparameter combination.

5. Given your optimal model found in the previous step, evaluate your model on the test set. Present your results.